

Product Requirements Document (PRD)

Product Name

Restaurant Knowledge OS

Working Tagline

Talk to your restaurant AI. It learns your business and answers customers in real time.

1. Product Summary

Restaurant Knowledge OS is a multilingual, self-hosted AI platform that allows restaurant owners to create their own business-specific AI assistant through natural conversation. Instead of requiring prompt engineering, manual FAQ uploads, or technical setup, the owner teaches the system in plain language. The platform converts these conversations into a structured, searchable restaurant knowledge base and uses that knowledge to answer customer questions through text or voice.

The product is designed first for restaurants and food businesses, then later expandable to other small businesses.

The system is intentionally built as a private knowledge platform rather than a model-training platform. The language model remains shared. Each restaurant has its own isolated knowledge, memory, settings, and conversation history.

2. Problem Statement

Small restaurants receive repeated customer queries every day about opening hours, menu items, pricing, delivery, reservation availability, offers, payment methods, and policies. These questions are simple but constant, and answering them consumes staff time.

Current AI chatbot builders are not ideal for small business owners because they often require:

- prompt writing
- technical configuration
- manual document upload
- dependency on developers
- expensive cloud APIs
- vendor lock-in

Most importantly, existing solutions do not give restaurant owners a simple way to teach or update their own AI assistant using everyday conversation.

Restaurant information also changes frequently. Menus, prices, timings, holidays, discounts, and delivery rules can change at any time. The AI must reflect these updates immediately without retraining a model.

The core problem is not model training. The core problem is business-specific knowledge acquisition, storage, validation, retrieval, and real-time response generation.

3. Product Vision

Enable every restaurant owner to create and continuously improve a personalized AI receptionist simply by talking to it.

The owner should be able to say things like:

- We open at 11 AM.
- Paneer Pizza costs ₹250.
- Sunday is closed.
- We deliver within 5 km.

The system should automatically understand, structure, store, and use this knowledge to answer customer questions in the correct language, with low latency, and without data mixing across restaurants.

4. Product Goals

Primary Goals

1. Allow restaurant owners to teach the AI using natural language.
2. Convert free-form conversation into structured restaurant knowledge.
3. Support multilingual customer interaction in Hindi, Gujarati, and English first.
4. Keep every restaurant's knowledge strictly isolated.
5. Answer customer queries using only that restaurant's private knowledge.
6. Support both text and voice interactions.
7. Keep the MVP self-hostable and low-cost.

Secondary Goals

1. Provide edit, delete, and versioning support for knowledge.
 2. Support future expansion to phone calls, WhatsApp, and additional Indian languages.
 3. Make the architecture production-ready without relying on proprietary cloud AI as the core dependency.
 4. Design the system so it can scale from one laptop to a multi-tenant SaaS.
-

5. Non-Goals for MVP

The following are explicitly out of scope for version 1:

- fine-tuning the base LLM
 - training a custom voice model
 - real telecom phone calling
 - WhatsApp integration
 - billing and subscription management
 - CRM
 - analytics dashboards beyond basic usage tracking
 - multi-agent orchestration
 - appointment booking engine with payment collection
 - inventory management
 - large-scale enterprise workflows
-

6. Target Users

6.1 Restaurant Owner / Admin

A non-technical owner or manager who wants a custom AI receptionist for their restaurant.

Needs:

- easy onboarding
- simple training flow
- ability to update knowledge
- ability to test AI responses
- visibility into stored facts

6.2 Customer

A restaurant customer who wants quick answers to questions about the restaurant.

Needs:

- fast response
- natural conversation
- Hindi / Gujarati / English support
- accurate answers

6.3 Internal Operator / Founder

The system maintainer who manages deployments, knowledge model updates, and scaling.

Needs:

- tenant isolation
 - debugging visibility
 - safe defaults
 - low operational cost
-

7. Core Product Concept

The platform has two main experiences:

A. Owner Training Experience

The restaurant owner talks to the system like a new employee. The system extracts facts and stores them as structured knowledge.

B. Customer Conversation Experience

A customer asks a question through text or voice. The system retrieves only that restaurant's knowledge and responds in the same language as the customer whenever possible.

The system does not “learn” by changing the base LLM. It learns by updating the restaurant's private knowledge base.

8. Key Product Principles

1. **Private by design** — no data mixing between restaurants.
 2. **Conversation first** — owners should not need prompts or technical setup.
 3. **Knowledge over model training** — facts are stored and updated, not the LLM.
 4. **Multilingual by default** — Hindi, Gujarati, and English first.
 5. **Self-hostable** — the core system should run locally or on low-cost infrastructure.
 6. **Production-minded** — architecture should be simple enough for MVP, but not a dead end.
-

9. User Stories

Owner Stories

- As a restaurant owner, I want to create my restaurant profile so that the AI knows which business it belongs to.
- As a restaurant owner, I want to speak naturally to the AI so that I can teach it my business details without using prompts.
- As a restaurant owner, I want to review extracted facts so that I can correct mistakes.

- As a restaurant owner, I want to update menu items and prices so that customers receive current information.
- As a restaurant owner, I want to test the AI so that I can verify its responses before sharing it publicly.

Customer Stories

- As a customer, I want to ask about opening hours so that I can know whether the restaurant is open.
- As a customer, I want to ask in Hindi or Gujarati so that I can use my preferred language.
- As a customer, I want quick and reliable answers so that I do not need to wait for staff.

Internal Stories

- As the system, I want to isolate each restaurant's data so that one customer's query never uses another restaurant's facts.
 - As the system, I want to validate new facts so that conflicting updates do not corrupt the knowledge base.
 - As the system, I want to log conversations safely so that owners can inspect interactions without leaking private data.
-

10. Functional Requirements

10.1 Restaurant Setup

- Create restaurant account
- Store restaurant name, type, language preference, timezone, location, and contact details
- Assign restaurant ID and owner ID
- Initialize private knowledge space

10.2 Knowledge Ingestion / Training

- Accept owner input via text
- Accept owner input via voice
- Transcribe voice to text
- Extract facts from conversation
- Categorize facts into structured objects
- Store extracted facts in the restaurant's knowledge base
- Allow owner to manually correct extracted facts
- Support incremental updates
- Support deletion and replacement of facts

10.3 Knowledge Organization

- Store knowledge by category such as:
 - business hours
 - menu items
 - prices

- offers
- delivery rules
- reservation rules
- payment methods
- holiday closures
- contact details
- policies
- Maintain version history for updates
- Flag conflicting facts

10.4 Customer Query Handling

- Accept text question
- Accept voice question
- Detect customer language
- Retrieve restaurant-specific context
- Generate answer using retrieved knowledge only
- Return answer as text
- Optionally convert answer to speech

10.5 Multilingual Support

- Support English, Hindi, and Gujarati in MVP
- Preserve language matching when possible
- Allow cross-language retrieval
- owner teaches in one language
- customer asks in another
- system still answers correctly

10.6 Admin and Review

- View all stored knowledge entries
- Edit or delete entries
- View recent conversations
- Test AI in sandbox mode
- Approve or reject uncertain extractions

11. Technical Requirements

11.1 Local / Self-Hosted Execution

The MVP must run locally on the founder's hardware and remain usable without mandatory proprietary cloud AI.

11.2 Hardware Constraint

The initial development system is:

- Intel i5-12450H
- RTX 4060 Laptop GPU with 8 GB VRAM
- 16 GB RAM

The design must be practical on this machine.

11.3 Performance Targets

- Knowledge extraction should complete within a few seconds for typical owner input
- Customer responses should feel near real-time
- Voice roundtrip should remain low latency for demo use

11.4 Scalability Targets

Architecture should support:

- 1 restaurant for demo
- 10 restaurants for MVP
- 100 restaurants for beta
- 1000+ restaurants for production
- future sharding for larger scale

11.5 Security Targets

- strict tenant isolation
- encrypted API traffic
- safe knowledge updates
- no cross-tenant retrieval
- role-based access for owner/admin

12. Proposed Technology Strategy

12.1 Core Stack

- Frontend: Next.js
- Backend: FastAPI
- SQL Database: PostgreSQL
- Vector Database: Qdrant
- Speech-to-Text: Faster Whisper
- Embeddings: multilingual-e5
- LLM: Qwen 3 8B or equivalent local multilingual model
- Text-to-Speech: AI4Bharat Indic-TTS or comparable open-source TTS
- Structured Output: Pydantic AI

- Deployment: Docker first, Kubernetes later

12.2 Why This Stack

- open-source friendly
- locally runnable
- multilingual
- production oriented
- low cost
- easy to debug
- scalable later

12.3 Design Preference

Use a hybrid approach:

- PostgreSQL for structured facts
- Qdrant for semantic retrieval

This avoids using vectors for everything and keeps simple facts fast and precise.

13. MVP Scope

MVP Features

1. Restaurant onboarding
2. Owner training via text
3. Owner training via voice
4. Knowledge extraction into structured JSON
5. Review/edit/delete knowledge
6. Customer text chat
7. Customer voice input/output in browser
8. Multilingual support for Hindi, Gujarati, and English
9. Private knowledge per restaurant
10. Basic test dashboard

MVP Success Criteria

- An owner can create a restaurant and train it without coding.
 - A customer can ask questions and receive correct answers.
 - Two restaurants cannot access each other's knowledge.
 - Knowledge updates reflect immediately.
 - Voice mode works well enough for a live demo.
-

14. Detailed User Flows

14.1 Owner Onboarding Flow

1. Owner signs up.
2. Owner creates a restaurant profile.
3. Owner selects preferred language(s).
4. Owner enters basic restaurant details.
5. System creates a private knowledge space.

14.2 Owner Training Flow

1. Owner opens training screen.
2. Owner types or speaks restaurant facts.
3. Speech is transcribed if voice is used.
4. Extraction engine identifies business facts.
5. System shows extracted structured records.
6. Owner approves or edits.
7. Approved knowledge is saved.

14.3 Customer Query Flow

1. Customer opens chat or voice interface.
2. Customer asks a question.
3. Speech is transcribed if voice is used.
4. System identifies restaurant tenant.
5. System retrieves relevant knowledge.
6. LLM composes answer using only retrieved context.
7. Answer is shown as text or spoken aloud.

14.4 Update Flow

1. Owner says a fact changed.
2. System identifies conflict or new data.
3. Existing fact is updated or versioned.
4. Knowledge index is refreshed.
5. Future answers use the latest data.

15. Knowledge Model

Each restaurant knowledge entry should be stored as a structured object.

Example Categories

- business_hours
- menu_item

- offer
- delivery_rule
- reservation_rule
- payment_policy
- holiday
- contact
- special_instruction
- faq

Example Schema

```
{
  "restaurant_id": "r_001",
  "category": "menu_item",
  "key": "paneer_pizza",
  "value": {
    "price": 250,
    "available": true
  },
  "source_language": "gujarati",
  "status": "active",
  "version": 3,
  "updated_at": "2026-07-12T10:00:00Z"
}
```

16. Architecture Requirements

16.1 Shared Components

- LLM service shared across restaurants
- STT service shared across restaurants
- TTS service shared across restaurants
- Embedding service shared across restaurants

16.2 Per-Tenant Components

- restaurant knowledge records
- conversation history
- memory records
- settings
- voice preferences
- access permissions

16.3 Isolation Strategy

Use restaurant_id on every relevant record. Use either:

- one Qdrant collection per restaurant, or
- one shared collection with tenant filtering

For MVP, collection-per-restaurant is simplest. For later scale, use shared collection + tenant_id filters.

17. Non-Functional Requirements

17.1 Usability

- owner should not need prompts
- training should feel like natural conversation
- interface should be simple and minimal

17.2 Reliability

- system should not mix data between restaurants
- extraction errors should be visible and editable
- system should fall back safely if confidence is low

17.3 Maintainability

- modular codebase
- clean API boundaries
- easy model replacement
- easy database migration

17.4 Cost Efficiency

- avoid unnecessary cloud dependencies
- support local inference
- scale incrementally only when revenue appears

17.5 Extensibility

- future support for phone calls
 - future support for WhatsApp
 - future support for more Indian languages
 - future support for additional verticals
-

18. Recommended Product Decisions

Decision 1: Do not fine-tune the LLM in MVP

Use retrieval and structured knowledge instead.

Decision 2: Use a hybrid SQL + vector architecture

SQL for facts, vector search for fuzzy retrieval.

Decision 3: Start with browser-based voice

Do not start with telecom phone numbers.

Decision 4: Keep the first version focused on restaurants only

Do not expand to all industries in the first release.

Decision 5: Optimize for knowledge capture, not model training

The product is about helping businesses create their own private business brain.

19. Risks and Mitigations

Risk: Knowledge conflicts

Mitigation: show conflict warnings and require owner confirmation.

Risk: Hallucinated answers

Mitigation: answer only from retrieved restaurant knowledge; use guardrails.

Risk: Cross-tenant leakage

Mitigation: tenant scoping everywhere; test with isolation cases.

Risk: Slow voice latency

Mitigation: use lightweight local models and keep initial voice demo browser-based.

Risk: Scope creep

Mitigation: freeze MVP and keep phone calls, WhatsApp, CRM, and analytics out of scope.

Risk: Overengineering early

Mitigation: start with a monolith, split into services only when required.

20. Success Metrics

Product Metrics

- number of restaurants onboarded
- number of knowledge entries created
- number of successful customer responses
- response accuracy
- response latency
- number of owner edits vs rejected extractions

Technical Metrics

- STT accuracy
- extraction precision
- retrieval precision
- hallucination rate
- tenant isolation failures
- average latency per request

Business Metrics

- number of paid conversions
 - retention rate
 - owner activation rate
 - customer query completion rate
-

21. Future Roadmap

Phase 1: MVP

- text-based training
- structured knowledge extraction
- customer text chat
- browser voice demo

Phase 2: Multilingual Enhancement

- improve Gujarati, Hindi, and English quality
- improve retrieval quality
- improve TTS voices

Phase 3: Production Readiness

- authentication hardening
- monitoring
- logs
- backups
- tenant scaling

Phase 4: Voice at Scale

- LiveKit or Pipecat integration
- real-time voice streaming
- better turn-taking and interruption handling

Phase 5: Call Automation

- phone call integration
- business-owned numbers
- telephony layer

Phase 6: Expansion

- cafés
- bakeries
- clinics
- salons
- retail stores
- coaches and local service providers

22. Final Product Definition

Restaurant Knowledge OS is not a generic chatbot builder. It is a multilingual private knowledge platform for restaurants that allows owners to teach an AI receptionist through natural conversation and allows customers to interact with that receptionist in real time through text or voice.

The product's fundamental asset is not the language model. The asset is the restaurant's structured private knowledge base.

23. Immediate Build Recommendation

Do not start with voice calls or production scaling.

Build in this order: 1. restaurant onboarding 2. knowledge schema 3. knowledge extraction 4. knowledge review/editing 5. customer text chat 6. browser voice demo 7. multilingual refinement 8. tenant isolation hardening 9. production hardening

That is the minimum path to a real product.